Coding Task

Write an application which converts a currency (dollars) from numbers into words.
The maximum number of dollars is 999 999 999.
The maximum number of cents is 99.
The separator between dollars and cents is a ‘,’ (comma).

Examples: Input	Expected output
0	zero dollars
1	one dollar
25,1	twenty-five dollars and ten cents
0,01	zero dollars and one cent
45 100	forty-five thousand one hundred dollars
999 999 999,99	nine hundred ninety-nine million nine hundred ninety-nine thousand nine hundred ninety-nine dollars and ninety-nine cents

The application must support at least two languages: English and one additional language of your choice (e.g., German). The user must be able to select the language, and the currency conversion output must reflect the selected language.

Non-functional requirements:

Choose between a desktop and a web application!

Desktop application	Web application
Use .NET 8.0+	Use ASP.NET for backend.
Use client server architecture.	Use client server architecture.
The client should have a GUI.	Use a modern frontend framework of your choice (e.g., React, Angular, Vue, or Blazor).
Conversion should be implemented on server side.	Conversion should be implemented on server side.

Submission Guidelines:

- **Repository:** The final solution should be provided as a GitHub repository.
- **Documentation:** Include a short README describing how to build and run the application, key design decisions and assumptions, and any limitations or known issues.
- **Architecture & Code Quality:** We value clean, maintainable, and well-structured code. Please ensure your solution reflects good engineering practices.
- **AI Usage (Transparency Requirement): If AI tools or assistance were used during development, please include the relevant prompt history as part of your submission to ensure transparency.**

Coding Task (Desktop)

Write an application which converts a currency (dollars) from numbers into words.
The maximum number of dollars is 999 999 999.
The maximum number of cents is 99.
The separator between dollars and cents is a ‘,’ (comma).

Examples: Input	Expected output
0	zero dollars
1	one dollar
25,1	twenty-five dollars and ten cents
0,01	zero dollars and one cent
45 100	forty-five thousand one hundred dollars
999 999 999,99	nine hundred ninety-nine million nine hundred ninety-nine thousand nine hundred ninety-nine dollars and ninety-nine cents

The application must support at least two languages: English and one additional language of your choice (e.g., German). The user must be able to select the language, and the currency conversion output must reflect the selected language.

Non-functional requirements:

- Create a windows desktop application.
- Use the latest stable version of .NET (e.g., .NET 8 or newer at the time of implementation).
- Use client-server architecture.
- The client should have a GUI.
- Conversion should be implemented on the server side.

Submission Guidelines:

- **Repository:** The final solution should be provided as a GitHub repository.
- **Documentation:** Include a short README describing how to build and run the application, key design decisions and assumptions, and any limitations or known issues.
- **Architecture & Code Quality:** We value clean, maintainable, and well-structured code. Please ensure your solution reflects good engineering practices.
- **AI Usage (Transparency Requirement): If AI tools or assistance were used during development, please include the relevant prompt history as part of your submission to ensure transparency.**

Coding Task (Web)

Write an application which converts a currency (dollars) from numbers into words.
The maximum number of dollars is 999 999 999.
The maximum number of cents is 99.
The separator between dollars and cents is a ‘,’ (comma).

Examples: Input	Expected output
0	zero dollars
1	one dollar
25,1	twenty-five dollars and ten cents
0,01	zero dollars and one cent
45 100	forty-five thousand one hundred dollars
999 999 999,99	nine hundred ninety-nine million nine hundred ninety-nine thousand nine hundred ninety-nine dollars and ninety-nine cents

The application must support at least two languages: English and one additional language of your choice (e.g., German). The user must be able to select the language, and the currency conversion output must reflect the selected language.

Non-functional requirements:

- Create a web application.
- Use the latest stable version of .NET (e.g., .NET 8 or newer at the time of implementation).
- Use ASP.NET for backend
- Use client-server architecture.
- Use a modern frontend framework of your choice (e.g., React, Angular, Vue, or Blazor).
- Conversion should be implemented on the server side.

Submission Guidelines:

- **Repository:** The final solution should be provided as a GitHub repository.
- **Documentation:** Include a short README describing how to build and run the application, key design decisions and assumptions, and any limitations or known issues.
- **Architecture & Code Quality:** We value clean, maintainable, and well-structured code. Please ensure your solution reflects good engineering practices.
- **AI Usage (Transparency Requirement): If AI tools or assistance were used during development, please include the relevant prompt history as part of your submission to ensure transparency.**